

```
# =====
# AI LEARNING & STUDY ASSISTANT
# RAG + MEMORY + TOOLS
# Google Colab - Simple Working Version
# =====

!pip -q install gradio scikit-learn

import gradio as gr
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# -----
# STUDY MATERIAL
# -----

study_material = [
    "Artificial Intelligence is a branch of computer science that enables machines to perform tasks that normally require human intelligence.",

    "Machine Learning is a subset of Artificial Intelligence. It allows computers to learn patterns from data and make predictions without being explicitly programmed.",

    "Supervised Learning uses labeled training data. Classification and regression are common examples of supervised learning.",

    "Unsupervised Learning uses unlabeled data. Clustering and dimensionality reduction are common examples.",

    "Deep Learning is a subset of Machine Learning that uses neural networks with multiple layers to learn complex patterns.",

    "Natural Language Processing, or NLP, allows computers to understand, process and generate human language.",

    "Retrieval Augmented Generation, or RAG, retrieves relevant information from documents and uses that information to generate better answers.",
```

```

    "AI memory allows an assistant to remember useful information from
previous conversations and use it in future interactions.",

    "AI tools allow an assistant to perform specific tasks such as
calculations, searching information, generating quizzes and creating
learning plans."
]

# -----
# RAG SETUP
# -----

vectorizer = TfidfVectorizer()
vectors = vectorizer.fit_transform(study_material)

def retrieve_answer(question):

    question_vector = vectorizer.transform([question])

    similarity = cosine_similarity(
        question_vector,
        vectors
    )[0]

    best_index = similarity.argmax()

    if similarity[best_index] < 0.05:
        return "Sorry, I could not find this topic in my study material."

    return study_material[best_index]

# -----
# MEMORY
# -----

memory = []

```

```

def ask_ai(question):

    if question.strip() == "":
        return "Please enter a question.", show_memory()

    answer = retrieve_answer(question)

    # Save memory
    memory.append({
        "question": question,
        "answer": answer
    })

    # Keep last 5 conversations
    if len(memory) > 5:
        memory.pop(0)

    return answer, show_memory()

def show_memory():

    if len(memory) == 0:
        return "🧠 No previous questions."

    text = "### 🧠 Recent Memory\n\n"

    for i, item in enumerate(memory):
        text += f"**{i+1}. Question:** {item['question']}\n\n"
        text += f"**Answer:** {item['answer']}\n\n"

    return text

def clear_memory():

    memory.clear()

    return "🧹 Memory cleared.", show_memory()

```

```

# -----
# LEARNING PLAN TOOL
# -----

def create_plan(topic, days):

    if topic.strip() == "":
        return "Please enter a topic."

    days = int(days)

    result = f"# 📖 {days}-Day Learning Plan\n\n"
    result += f"### Topic: {topic}\n\n"

    for day in range(1, days + 1):

        if day == 1:
            task = "Learn the basic concepts."
        elif day == 2:
            task = "Study important definitions and examples."
        elif day == 3:
            task = "Practice basic questions."
        elif day == 4:
            task = "Study practical applications."
        elif day == 5:
            task = "Solve practice problems."
        elif day == 6:
            task = "Revise difficult concepts."
        else:
            task = "Revision and self-test."

        result += f"### Day {day}\n"
        result += f"📖 {task}\n"
        result += f"🕒 Study: 1 hour\n\n"

    return result

# -----

```

```

# QUIZ TOOL
# -----

def generate_quiz(topic):

    context = retrieve_answer(topic)

    if "Sorry" in context:
        return "No study material found for this topic."

    quiz = f"""
# 🧩 Quiz

### Topic
{topic}

### Question 1
What is the main concept discussed in the study material?

**Answer:**
{context}

---

### Question 2
Explain this topic in your own words.

**Answer:**
{context}

---

### Question 3
Why is this concept important?

**Answer:**
It is important because it helps understand and apply the concept in
real-world AI systems.
"""

```

```

return quiz

# =====
# WEB INTERFACE
# =====

with gr.Blocks(title="AI Learning & Study Assistant") as app:

    gr.Markdown("""
# 🎓 AI Learning & Study Assistant

### RAG + Memory + Tools

Ask questions, create learning plans and generate quizzes.
""")

    # -----
    # ASK AI
    # -----

    with gr.Tab("💬 Ask AI"):

        question = gr.Textbox(
            label="Ask your question",
            placeholder="Example: What is Machine Learning?"
        )

        ask_button = gr.Button(
            "🔍 Generate Answer"
        )

        answer = gr.Markdown(
            label="AI Answer"
        )

        memory_output = gr.Markdown()

        ask_button.click(
            ask_ai,

```

```

        inputs=question,
        outputs=[answer, memory_output]
    )

# -----
# LEARNING PLAN
# -----

with gr.Tab("📖 Learning Plan"):

    topic = gr.Textbox(
        label="Enter Topic",
        placeholder="Example: Artificial Intelligence"
    )

    days = gr.Number(
        label="Number of Days",
        value=7,
        precision=0
    )

    plan_button = gr.Button(
        "🚀 Create Plan"
    )

    plan_output = gr.Markdown()

    plan_button.click(
        create_plan,
        inputs=[topic, days],
        outputs=plan_output
    )

# -----
# QUIZ
# -----

with gr.Tab("🧩 Generate Quiz"):

    quiz_topic = gr.Textbox(

```

```

        label="Enter Quiz Topic",
        placeholder="Example: Deep Learning"
    )

    quiz_button = gr.Button(
        "
```

```
        clear_memory,  
        outputs=[clear_status, memory_screen]  
    )  
  
# -----  
# START  
# -----  
  
print("🚀 Starting AI Learning & Study Assistant...")  
print("✅ No API key required")  
print("✅ No file upload required")  
print("✅ No ngrok required")  
  
app.launch(share=True)
```